

Danmarks
Tekniske
Universitet



JPEG200 for Digital Cinema Implementation of a Wavelet-Based Image Encoder

AUTHOR

Leonor Castelo
s257446

May 15, 2026

Contents

1	Introduction	iii
1.1	Scope and Simplifications	iii
2	Coding Scheme: T-Q-E Block Diagram	iii
3	Block T: Discrete Wavelet Transform	iv
3.1	Theoretical Background	iv
3.2	Wavelet Filter: CDF 9/7	v
4	Block Q: Scalar Quantization	vi
4.1	Theoretical Background	vi
4.2	Deadzone Quantizer	vi
4.3	Per-Subband Step Sizes	vi
5	Block E: Entropy Coding	vii
5.1	Theoretical Background	vii
5.2	Adaptive Arithmetic Coding	vii
5.2.1	Algorithm	viii
5.2.2	Symbol Preparation	viii
5.2.3	E1/E2/E3 Rescaling	viii
5.3	Entropy Estimation: Adaptive H1 context Model	ix
5.4	Bit-Rate Computation and Compression Factor	ix
5.5	Comparison: Arithmetic Code Length vs H1 Estimate	ix
6	Evaluation Metrics	x
6.1	Bit-Rate Metrics	x
6.1.1	Bits per Pixel (bpp)	x
6.1.2	Compression Factor (CF)	x
6.1.3	Redundancy	x
6.2	Entropy and Cross-Entropy Estimation	xi
6.2.1	Shannon Entropy Bound	xi
6.2.2	Order-1 Cross-Entropy (H1)	xi
6.3	Objective Quality Metrics	xi
6.3.1	Mean Square Error (MSE)	xi
6.3.2	Peak Signal-to-Noise Ratio (PSNR)	xi
6.4	Rate-Distortion Analysis	xii
7	Results	xii
7.1	Numerical Summary	xii
7.2	Rate-Distortion Curve	xiii
7.3	Arithmetic Code Length vs. H1 Estimate	xiv
7.4	Objective Quality: PSNR	xiv
7.5	Subjective Quality Assessment	xv

8 Conclusion	xv
A Project Description	xviii
A.1 Introduction	xviii
A.2 Relation to the T-Q-E Block Diagram	xviii
A.3 Evaluation Plan	xviii
B Implementation Details	xix
B.1 Forward and Inverse Discrete Wavelet Transform	xix
B.2 Quantization	xxi
B.3 Adaptive Arithmetic Coding	xxii
B.4 Entropy estimation	xxiv
B.5 Main processing loop	xxv
C User Guide	xxvii
C.1 Requirements	xxvii
C.2 Running the code	xxvii
C.3 Key Parameters	xxvii
C.4 Output Interpretation	xxviii
D Statement on use of AI Tools	xxviii

1 Introduction

The Digital Cinema standard mandates JPEG2000 as the sole compression format for digital film distribution worldwide [1]. Cinema projectors must decode compressed bit-streams at resolution up to 4096×2160 (4K) at 24 frames per second, targeting bitrates up to 250 Mbit/s while maintaining PSNR above 45 dB [15]. Archiving this requires three essential properties absent from older standards such as JPEG, multi-resolution scalability and near-lossless quality at moderate compression ratios, absent from older standards such as JPEG.

1.1 Scope and Simplifications

The full JPEG 2000 standard [3] is highly complex. This project implements the fundamental transform and quantization blocks in full, while substituting the EBCOT entropy coder with two estimations that bound the achievable bit-rate. Table1 summarizes the key differences.

Component	Full JPEG 2000	This Implementation
Entropy Coder	EBCOT: MQ-coder, bit-plane coding [4]	Not implemented, replaced by an arithmetic coder
Bit-rate measure	Actual compressed bit-stream length	Sparsity-based code length + cross-entropy estimate
Wavelet Filter	CDF 9/7 irreversible	CDF 9/7 irreversible (same)
DWT Levels	5-6 levels (DCI profile)	3 levels (10 subbands)
Test Image	4K cinema frames (4096×2160)	512×512 Lenna (standard test image)

Table 1: Comparison of full JPEG 2000 and this implementation

2 Coding Scheme: T-Q-E Block Diagram

The entire encoder can be summarized in three sequential blocks:

The inverse process (decoder) applies $E^{-1} \rightarrow Q^{-1} \rightarrow T^{-1}$ to reconstruct the image. Because T and E are lossless, the reconstructed image quality depends exclusively on block Q (quantization). The quantization step size δ is therefore the single parameter that controls the rate-distortion trade-off.

In Figure 1 we can see the general block diagram of the JPEG 2000 encoder and decoder.

Block	Name	Function
T	Transform (2D DWT)	Decorrelates the image and concentrates energy into a small number of low-frequency coefficients [6]
Q	Quantization	Deadzone Scalar Quantization. Reduces wavelet coefficients to signed integers. The sole source of distortion, controls the rate-distortion trade-off
E	Entropy Coding	Represents the quantized integer indices with the minimum number of bits using arithmetic coding

Table 2: The T-Q-E encoder blocks

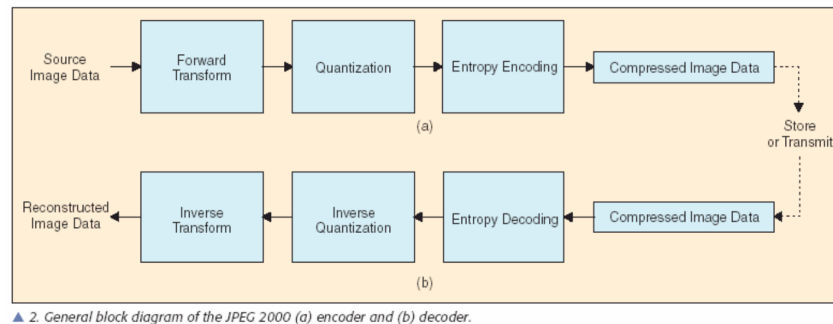


Figure 1: JPEG 2000 block diagram

3 Block T: Discrete Wavelet Transform

3.1 Theoretical Background

The 2D Discrete Wavelet Transform decomposes an image into a hierarchy of frequency subbands. The key property exploited by JPEG 2000 is energy compaction. After the DWT, most image concentrates in a small number of low-frequency (LL) coefficients, while the majority of high frequency coefficients are near zero [6]. This sparsity is what makes wavelet-based compression effective, sparse coefficients can be quantized aggressively with little perceptual impact.

The 1D DWT applies conjugate-mirror filter pairs (low-pass h_0 and high-pass h_1) and downsamples by two, producing two half-length-sub-signals. The 2D DWT exploits separability. The 1D DWT is applied first along all columns, yielding four subbands per level ??.

JPEG 2000 applies the DWT recursively to the LL subband for N_L levels producing $3N_L + 1$ subbands in total[17]. This project uses $N=3$, giving 10 subbands. The full DCI profile typically uses 5 levels, but 3 levels is sufficient to demonstrate all key phenomena on 512×512 images [3].

Subband	Frequencies	Content
LL	Low $\times$ Low	Reduced version of the original image (the most important because it contains most of the energy)
LH	Low $\times$ High	Horizontal edges
HL	High $\times$ Low	Vertical edges
HH	High $\times$ High	Diagonal details and fine textures. Least important.

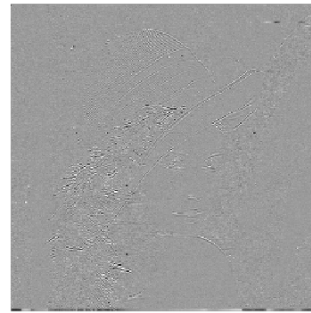
3.2 Wavelet Filter: CDF 9/7

The filter used is the Cohen-Daubechies-Feauveau (CDF) 9/7 wavelet, the standard for JPEG 2000 lossy compression [3]. In MATLAB it is available as 'bio4.4'. Its key properties are excellent energy compaction and near-linear phase (important for visual quality). The extension mode is set to periodization ('per'), which treats the signal as periodic and avoids boundary artefacts [13][14].

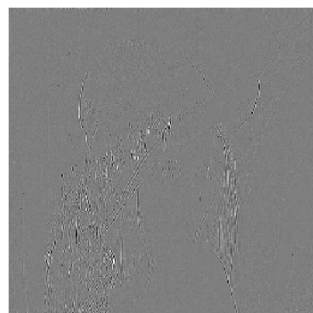
For the results, we have the example of the lena image:



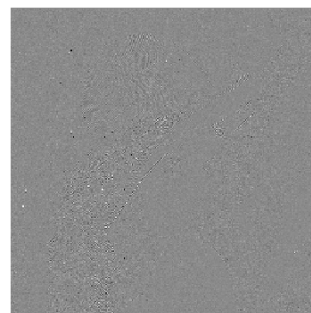
(a) LL (Approximation)



(b) LH (Horizontal)



(c) HL (Vertical)



(d) HH (Diagonal)

Figure 2: Decomposition of the lena image using the discrete wavelet transform

4 Block Q: Scalar Quantization

4.1 Theoretical Background

A scalar quantizer $Q()$ maps a real-valued input to a discrete index [5]. Quality is measured by the Mean-Square Error (MSQE):

$$\sigma_q^2 = \sum_{i=1}^M \int_{b_{i-1}}^{b_i} (x - y_i)^2 f_X(x) dx$$

where b_i are the decision boundaries and y_i the reconstruction levels.

The entropy of the quantizer output $H(Q)$ gives the minimum bit-rate a lossless entropy coder can achieve for a given quantization. The theoretical rate-distortion bound for a Gaussian source is [7]:

$$R(D) = \frac{1}{2} \log_2(\sigma^2/D) \text{ bits/sample}$$

Wavelet coefficients follow a near-Laplacian distribution [6] motivating the use of a deadzone quantizer rather than a Gaussian-optimized Lloyd-Max design.

4.2 Deadzone Quantizer

The JPEG2000 standard specifies a uniform deadzone quantizer for the irreversible (9/7) path [3].

The deadzone is the central interval $(-\delta, \delta)$ of width 2δ , twice the width of all other intervals. This exploits the Laplacian shape of wavelet coefficients. The majority fall near zero and can be mapped to 0 without significant distortion.

Forward quantization:

$$q = Q(x) = \text{sign}(x) \left\lfloor \frac{|x|}{\delta} \right\rfloor$$

Reconstruction (midpoint rule):

$$\hat{x} = Q^{-1}(q) = \begin{cases} 0, & \text{if } q = 0 \\ \text{sign}(q) (|q| + 0.5) \delta, & \text{otherwise} \end{cases}$$

The +0.5 offset places each reconstruction point at the midpoint of its interval, minimizing the MSQE for a locally uniform distribution within the interval [5].

4.3 Per-Subband Step Sizes

Rather than applying a single uniform step size δ to all subbands, this implementation uses a heuristic per-subband scaling that reflects the energy decay across decomposition levels and perceptual sensitivity:

Subband	Step Size	Justification
LL	$\delta \times 0.5$	Highest-energy subband so quantization errors are highly visible. Finer quantization justified at cost of more bits.
LH, HL (level j)	$\delta \times 2^{j-1}$	Scaled with level, edges less important at high frequency
HH (level j)	$\delta \times 2^{j-1} \times 1.5$	Diagonal details, less perceptible to the human eye

The $2^{(j-1)}$ scalling for LH/HL is consistent with the bit-allocation principles of rate-distortion theory [5]. Each additional DWT level halves subband dimesions and roughly halves typical coefficient magnitudes, so doubling δ maintains approximately constant quantization noise variance relative to subband energy. In the full JPEG 2000 specification, per subbandstep sizer are derived from fixed-point arithmetic on the CDF 9/7 filter gain [3]. This implementation however approximates that behaviour heuristically.

5 Block E: Entropy Coding

5.1 Theoretical Background

The entropy coding block is the final stage of the encoder. Its purpose is to map the integer-value quantized wavelet coefficients, produce d by block Q, to a compact binary bitstream, exploiting the statistical redundancy in the symbol sequence. While block T concentrates image energy and block Q introduces the sole source of distortion, block E determines how efficiently the resulting discrete symbols can be represented.

The fundamental limit on compression efficiency is Shannon's source coding theorem [7]. For a memoryless source with symbol probabilities $\{p_i\}$, the average code length per symbol is lower-bounded by the entropy:

$$H(X) = - \sum_i p_i \log_2 p_i \text{ bits/symbol}$$

In the full JPEG 2000 standard, the entropy coder is the Embedded Block Coding with Optimized Truncation (EBCOT algorithm [4], which operates on bit-planes and uses 17 spatial context labels from a 2D neighborhood to predict each bit [16][17]. EBCOT processes coefficients bit-plane by bit-plane, from the most significant bit to the least significant, using a context-adaptive binary arithmetic coder known as the MQ-coder. This implementation replaces EBCOT with a simpler adaptive arithmetic coder, retaining the fundamental entropy-coding principle while abstracting the spatial context modelling.

5.2 Adaptive Arithmetic Coding

Arithmetic coding is a method for losslessly encoding a sequence of symbols from a source alphabet. Unlike Huffman coding, arithmetic coding is not restricted to integer-

length codewords per symbol and can approach the entropy limit to within a fraction of one bit per sequence [5],[16].

The encoder maps the entire input sequence to a single fractional number in $[0,1)$. At each step, the current interval $[low,high)$ is subdivided proportionally to the symbol probabilities, and the sub-interval corresponding to the observed symbol is retained. The final interval can be represented by any number within it, typically the shortest binary fraction.

5.2.1 Algorithm

This implementation uses a 32-bit fixed-precision arithmetic coder with rescaling (E1/E2/E3 condition handling) to avoid underflow [16]. The key parameters are:

- Precision: 32-bit integers for the interval boundaries $[low, high]$;
- Probability model: adaptive order-0 (symbol frequencies updated after each symbol);
- REscalling: hal-scaling (E1/E2) and quarter-scaling (E3) with pending-bits counter;
- Frequency rescaling: halved when the total frequency count exceeds 10^5 to prevent integer overflow and maintain adaptability.

5.2.2 Symbol Preparation

Before encoding, all quantized subband coefficients are concatenated into a single sequence. The minimum symbol value is subtracted to shift the alphabet to start at 1, and a symbol is appended.

Shifting to a 1-based alphabet is required because the arithmetic coder uses symbol indices directly as array indices into the frequency table.

5.2.3 E1/E2/E3 Rescaling

Without rescaling, the interval $[low,high]$ would converge to zero as more symbols are encoded, causing loss of precision. Three rescaling conditions are applied after each symbol:

- E1 (upper half): if $high < HALF$, output 0 and pending bits;
- E2 (lower half): if $low \geq HALF$, output 1 and pending bits. Subtract $HALF$, then scale;
- E3 (middle convergence): if $Quarter \leq low$ and $high < Three_QRT$, increment pending counter and subtract $Quarter$.

The pending bits mechanism handles E3 expansions. When the first E1 or E2 condition is met after one or more E3 conditions, the complement of the final bit is output for each pending expansion.

5.3 Entropy Estimation: Adaptive H1 context Model

In parallel, the code length is also estimated using an adaptive order-1 context model (H1) as a lower bound on what an ideal first-order coder could achieve [16]. The context is the binary sign of the previous coefficient and symbol probabilities are estimated with Laplace smoothing:

$$\hat{p}(x_k = i | c_k) = \frac{n + \delta}{N + |A|\delta}$$

The estimated code length is computed per subband and summed:

$$L_{H1}(x^n) = \sum_{k=0}^n -\log \hat{p}(x_k | c_k)$$

The H1 estimate is consistently lower than the arithmetic code length because it uses a first-order context that partly captures inter-symbol dependencies which the order-0 arithmetic coder ignores and it computes the theoretical entropy without the overhead of finite-precision interval arithmetic.

5.4 Bit-Rate Computation and Compression Factor

The arithmetic encoder outputs a binary bitstream. The bit-rate in bits per pixel is

$$\text{bpp_arith} = \frac{\text{length}(\text{bitstream})}{H \times W}$$

where $H \times W = 512 \times 512$ pixels for the Lenna image. The compression factor (CF) relative to the uncompressed 8-bpp baseline is:

$$\text{CF} = \frac{8}{\text{bpp_arith}}$$

The H1 estimate uses the same denominator but derives bits from the cross-entropy sum rather than an actual bitstream.

5.5 Comparison: Arithmetic Code Length vs H1 Estimate

The gap between the arithmetic code length and L_{H1} quantifies two sources of inefficiency in the arithmetic coder:

- Model order: The arithmetic coder uses an order-0 model, while L_{H1} uses a first-order context based on the sign of the previous coefficient. First-order dependencies in wavelet subbands are substantial, especially in high-energy subband;
- Finite precision: the 32-bit precision arithmetic introduces small interval rounding errors that accumulate over long sequences, adding a sub-bit overhead per symbol.

In the full JPEG2000 standard, EBCOT addresses the first inefficiency by using a 17 context labels derived from the 2D spatial neighborhood of each coefficient bit-plane, achieving rates 20 – 30% below the first-order on typical images [4]

6 Evaluation Metrics

This section describes all metrics used to assess the encoder’s performance: bit-rate measures, entropy estimation and objective image quality. Each subsection presents the theoretical definition, the MATLAB implementation and interpretation guidelines.

6.1 Bit-Rate Metrics

6.1.1 Bits per Pixel (bpp)

The primary bit-rate measure is bits per pixel (bpp), defined as the total number of bits in the compressed representation divided by the number of pixels in the image:

$$\text{bpp} = \frac{B_{\text{total}}}{H \times W}$$

For the Lenna image ($H=W=512$), the uncompressed baseline is 8 bpp (one byte per pixel in 8-bit grayscale). Two bpp values are reported for each δ . `bpp_arith` from the actual arithmetic bitstream and `bpp_H` from the cross-entropy estimate.

6.1.2 Compression Factor (CF)

The compression factor expresses how many times smaller the compressed file is compared to the uncompressed 8-bpp baseline:

$$\text{CF} = \frac{8}{\text{bpp}}$$

CF is a practical metric for DCI compliance checks. The DCI 1.5 bpp target corresponds to a CF of approximately $5.3 \times$.

6.1.3 Redundancy

The redundancy measures the fraction of bits in the arithmetic bitstream than a better entropy model (here, the H1 estimate) would eliminate:

$$\text{Redundancy} = \frac{B_{\text{arith}} - B_{\text{H1}}}{B_{\text{arith}}} \times 100$$

this metric directly quantifies the compression gain available from improved context modelling.

6.2 Entropy and Cross-Entropy Estimation

6.2.1 Shannon Entropy Bound

Shannon's source coding theorem establishes a fundamental lower bound on the average code length per symbol for any lossless code [7]. For a discrete random variable X :

$$H(X) = - \sum_{i=0}^n p(x_i) \log_2(p(x_i))$$

No uniquely decodable lossless code can achieve an average code length less than $H(X)$. The Shannon entropy is therefore the ultimate benchmark against which arithmetic coding and the H1 estimate are compared.

6.2.2 Order-1 Cross-Entropy (H1)

Rather than computing the true conditional entropy $H(X|X_{\text{prev}})$, the H1 metric estimates it using empirical frequency counts with Laplace smoothing. This choice makes the estimate robust to low-count symbols while remaining a tight approximation of the true conditional entropy for large subbands.

This implementation processes each subband independently. For each coefficient, the context c is the binarized sign of the previous coefficient. And the total H1 estimate is the sum across all 10 subbands.

The Laplace smoothing parameter $\alpha = 1$ is standard for entropy estimation tasks. It assigns a small non-zero probability to unobserved symbols, preventing $\log_2(0)$ singularities and slightly regularizing the estimate towards uniform for small alphabets. For large subbands the smoothing effect is negligible.

6.3 Objective Quality Metrics

6.3.1 Mean Square Error (MSE)

The Mean Square Error between the original image x and the reconstructed image $\hat{x}$ is:

$$\text{MSE} = \frac{\sum_{\{i,j\} \in N^2} x[i,j] - \hat{x}[i,j]}{H * W}$$

MSE is the mean squared quantization error per pixel. Because all distortion originates exclusively from block Q, MSE is a direct measure of quantization noise. MSE is a useful absolute distortion measure. However, it does not align well with human visual perception. Equal MSE at low and high spatial frequencies produces very different perceptual quality. This motivates reporting PSNR alongside MSE.

6.3.2 Peak Signal-to-Noise Ratio (PSNR)

PSNR is derived from MSE and expresses the ratio of the maximum possible signal power to the noise power in logarithmic units:

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}} \right)$$

where MAX=255 for 8-bit images.

The following table summarizes the commonly used PSNR thresholds for image quality assessment, used in this report to classify each operating point:

PSNR (dB)	Quality class	Context
> 50 dB	Indistinguishable from original	Lossless or almost lossless
40-50 dB	Excellent	DCI Digital Cinema (typically 45+ dB)
35-40 dB	Good	High-quality streaming
30-35 dB	Acceptable	Medium-quality streaming
< 30 dB	Noticeable degradation	Low bitrate, visible artefacts

6.4 Rate-Distortion Analysis

Rate-distortion theory provides the theoretical framework for interpreting the trade-off between bit-rate and reconstruction quality [7]. For a source with variance σ^2 and a squared-error distortion measure, the Shannon rate-distortion function is:

$$R(D) = \max\left\{0, \frac{\log_2(\sigma^2/D)}{2}\right\}$$

This function defines the minimum achievable rate for a given distortion D. The R-D curve of a practical codec, obtained by sweeping δ and plotting (bpp,PSNR) pairs, should lie above and to the right of the theoretical R(D) curve, with the gap indicating the coding inefficiency.

The two curves correspond to the two bit-rate measures. The arithmetic curve shows the actual encoder performance and the H1 curve shows the theoretical optimum of a first-order context coder. The horizontal distance between the two curves at any PSNR value is the bit-rate reduction available from improved context modelling.

7 Results

7.1 Numerical Summary

Table 3 presents the complete results for the 512×512 Lenna image across all seven step sizes. The uncompressed image is 8 bpp. The arithmetic coding results are produced by the actual bitstream encoder and the H1 values are the theoretical entropy estimates.

δ	bpp (arith)	bpp (H1)	CF (arith)	CF (H1)	Redundancy	PSNR (dB)	MSE
1	3.5751	3.2534	2.24	2.46	9%	50.51	0.5787
4	1.3198	1.0882	6.06	7.35	17.55%	39.81	6.7974
8	0.7244	0.5353	11.04	14.94	26.10%	35.73	17.3781
16	0.4184	0.2676	19.12	29.89	36.03%	31.90	42.00
32	0.2578	0.1345	31.03	59.49	47.85 %	28.63	89.10
64	0.1949	0.0826	41.05	96.90	57.64 %	26.56	143.53
128	0.1746	0.0650	45.82	123.13	62.79 %	26.23	155.05

Table 3: Full results for Lenna(512 × 512)

7.2 Rate-Distortion Curve

Figure 3 shows the rate-distortion curve for the 512 × 512 Lenna image, plotting PSNR (dB) as a function of bit-rate (bits per pixel) for both the arithmetic coder (blue) and the H1 cross-entropy estimate (red), across the seven quantization step sizes $\delta \in \{1, 4, 8, 16, 32, 64, 128\}$.

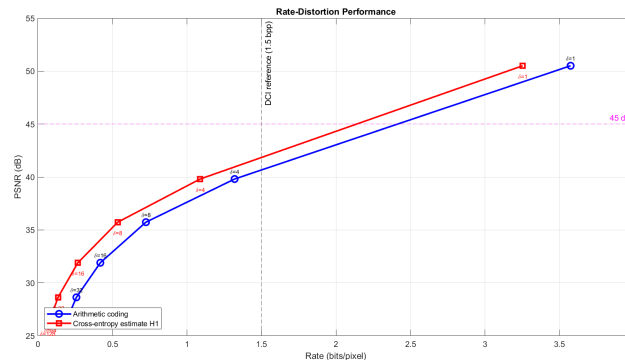


Figure 3: Rate-Distortion Curve

Both curves follow the expected concave shape of a rate-distortion function because PSNR increases steeply at low bit-rates and gradually flattens as the bit-rate grows. This is consistent with Shannons’s rate-distortion theory [7], which predicts diminishing quality returns per additional bit at high rates. The steep region corresponds to $\delta \geq 16$, where coarse quantization discards most wavelet coefficients and each reduction in δ recovers significant energy. The flat region at $\delta \leq 4$ corresponds to near-lossless operation, where most coefficients are already well-represented and additional bits provide marginal perceptual gain.

The vertical dashed line at 1.5 bpp marks the DCI reference bit-rate for digital cinema distribution [1]. At this operating point:

- The arithmetic coding curve intersects near $\delta = 4$, yielding PSNR ≈ 39.8 dB, which falls approximately 5 dB short of the 45 dB DCI quality target (horizontal dashed line);
- The H1 curve crosses 1.5 bpp at a PSNR between $\delta = 4$ and $\delta = 8$, suggesting that a first-order context coder would approach or reach 45 dB at this rate.

The rate-distortion curve confirms that the T-Q-E pipeline correctly implements the fundamental compression trade-off. The H1 estimate consistently outperforms the arithmetic coder, with the gap growing monotonically from 9% to 63% as δ increases, quantifying the benefit of context-adaptive entropy coding. Neither curve meets the DCI target of 45 dB at 1.5 bpp simultaneously, a gap explained by the simplified entropy coder and the reduced number of DWT levels relative to the full JPEG2000 DCI profile.

7.3 Arithmetic Code Length vs. H1 Estimate

The arithmetic code length consistently exceeds the H1 entropy across all δ , as expected. The redundancy (percentage gap) increases monotonically from 9% at $\delta = 1$ to 62.79 % at $\delta = 128$:

- At $\delta = 1$, the redundancy is lowest. Coefficients are dense and span a wide magnitude range, so the order-0 arithmetic coder's probability model is already a reasonable approximation of the true distribution. The H1 context model's first-order conditioning provides only marginal additional efficiency;
- As δ increases, sparsity grows rapidly (most coefficients become zero) and the alphabet concentrates around 1. The order-0 arithmetic coder struggles more because it must maintain a flat prior over a wide alphabet even though few symbols survive. The H1 model's context conditioning increasingly exploits the spatial correlations in the sparse nonzero pattern;
- At $\delta = 128$, the redundancy reaches 62.79%. The surviving nonzero coefficients are very sparse and tend to be large in magnitude, but the arithmetic coder still allocates bits across the full alphabet width, resulting in significant waste.

7.4 Objective Quality: PSNR

PSNR ranges from 50.51 dB at $\delta = 1$ (near-lossless) to 26.23 at $\delta = 128$. The following table 5 maps each result to a quality class.

δ	PSNR (dB)	Quality Class	Notes
1	50.51	Indistinguishable	Near-lossless. Exceeds DCI target (> 45 dB)
4	39.81	Excellent	Cinema-quality range
8	35.73	Good	High-quality streaming range
16	31.90	Acceptable	Medium-quality streaming
32	28.63	Visible artefacts	Below cinema requirements
64	26.56	Poor	Clear ringing and blocking artefacts
128	26.23	Very Poor	A lot of blemishes

Table 4: PSNR classification against DCI benchmarks

7.5 Subjective Quality Assessment

Figure 4 shows the reconstructed images at $\delta = 1$ until $\delta = 128$ respectively, alongside the original.

The first image is at $\delta = 1$ and it is visible indistinguishable from the original. But as we go along, for example, when $\delta = 8$ you start seeing slight texture blurring on close inspection, but the edges remain sharp. But after $\delta = 64$, there is visible ringing around edges and wavelet tiling patterns in smooth regions.

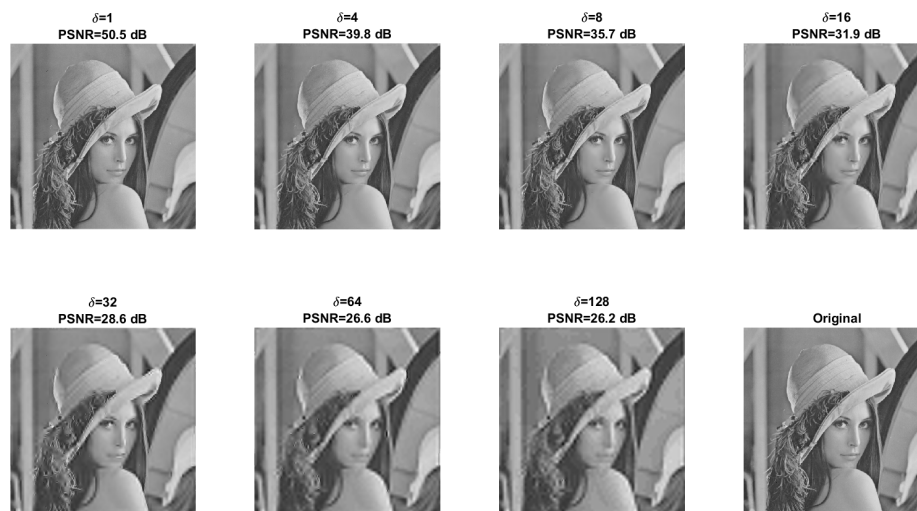


Figure 4: All reconstructive images ($\delta = 1$ to 128) plus the original

8 Conclusion

This project implemented and evaluated a simplified JPEG 2000 image encoder covering the complete T-Q-E pipeline, with an adaptive arithmetic entropy coder and a parallel H1 cross-entropy estimate reported for comparison. The key findings are:

- The arithmetic code length exceeds the H1 entropy estimate by 9-63% depending on the δ value. The gap grows with δ since the order-0 arithmetic coder cannot exploit the first-order spatial correlations that the H1 context model captures;
- PSNR at $\delta = 4$ is 39.81 dB below the DCI target. This gap can be attributed to the simplified 3-level DWT and heuristic step-size scaling;
- The rate-distortion curve correctly captures the expected trade-off since we have steep improvement from $\delta = 1$ to $\delta = 8$, followed by flattening and PSNR plateau above $\delta = 64$.

References

- [1] Digital Cinema Initiatives, LLC, “DCI Digital Cinema System Specification,” Version 1.4.1, 2021. [Online]. Available: <https://www.dcinovies.com/specification/> [Accessed: May 2025].
- [2] M. Antonini, M. Barlaud, P. Mathieu, and I. Daubechies, “Image coding using wavelet transform,” *IEEE Transactions on Image Processing*, vol. 1, no. 2, pp. 205–220, Apr. 1992.
- [3] ISO/IEC 15444-1:2019, *Information Technology — JPEG 2000 Image Coding System — Part 1: Core Coding System*. Geneva: International Organization for Standardization, 2019.
- [4] D. S. Taubman, “High performance scalable image compression with EBCOT,” *IEEE Transactions on Image Processing*, vol. 9, no. 7, pp. 1158–1170, Jul. 2000.
- [5] K. Sayood, *Introduction to Data Compression*, 5th ed. Morgan Kaufmann, 2018.
- [6] D. Taubman and M. W. Marcellin, *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Kluwer Academic Publishers, 2002.
- [7] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006.
- [8] A. Said and W. A. Pearlman, “A new, fast, and efficient image codec based on set partitioning in hierarchical trees,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 6, no. 3, pp. 243–250, Jun. 1996.
- [9] I. H. Witten, R. M. Neal, and J. G. Cleary, “Arithmetic coding for data compression,” *Communications of the ACM*, vol. 30, no. 6, pp. 520–540, Jun. 1987.
- [10] W. B. Pennebaker and J. L. Mitchell, *JPEG: Still Image Data Compression Standard*. Van Nostrand Reinhold, 1993.
- [11] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [12] S. Mallat, *A Wavelet Tour of Signal Processing: The Sparse Way*, 3rd ed. Academic Press, 2008.
- [13] MathWorks, “`wfilters` — Wavelet decomposition filters,” MATLAB R2024b Documentation. [Online]. Available: <https://www.mathworks.com/help/wavelet/ref/wfilters.html> [Accessed: May 2025].
- [14] MathWorks, “`dwtmode` — Discrete wavelet transform extension mode,” MATLAB R2024b Documentation. [Online]. Available: <https://www.mathworks.com/help/wavelet/ref/dwtmode.html> [Accessed: May 2025].

-
- [15] C. Christopoulos, A. Skodras, and T. Ebrahimi, “The JPEG 2000 still image coding system: an overview,” *IEEE Transactions on Consumer Electronics*, vol. 46, no. 4, pp. 1103–1127, Nov. 2000.
 - [16] S. Forchhammer, “Arithmetic Coding (Chapter 4),” Lecture slides, 34240 Data Science, Compression and Image Communication, DTU Electro, Feb. 2023.
 - [17] S. Forchhammer, “Wavelet and JPEG 2000 Coding (Chapter 15),” Lecture slides, 34240 Data Science, Compression and Image Communication, DTU Electro, Mar. 2023.
 - [18] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, Apr. 2004.

A Project Description

A.1 Introduction

The goal of this project is to implement and evaluate a simplified JPEG2000 image coder targeting the Digital Cinema use case, where JPEG2000 is the mandatory compression format, used for distributing feature films to cinemas worldwide at resolutions up to 4K and bitrates up to 500 Mbit/s. The project will analyse and implement the full compression pipeline using the T-Q-E block diagram as the guiding framework.

A.2 Relation to the T-Q-E Block Diagram

T- Transformation: 2D Discrete Wavelet Transform (DWT) that decorrelates spatial redundancy and produces a multi-resolution diagram (LL,LH,HL,HH).

Q- Quantization: Scalar quantizer applied independently per subband, with variable step sizes δb .

E - Entropy Coding: This is the hardest part. JPEG2000 uses EBCOT, a bit-plane coder with three coding passes per bit-plane and an adaptive arithmetic coder (MQ-coder). Due to its complexity, a simplified binary arithmetic coder will be implemented. The report will include a detailed analytical treatment of EBCOT and a comparison with the simplified implementation.

Deliberate Simplifications

1. Simplified binary arithmetic coder instead of full EBCOT/MQ-coder;
2. Using standard test images instead of 4K cinema frames.

A.3 Evaluation Plan

1. **Code Length:** The compressed output will be measured in bits per pixel for each test image, across a range of quantization step sizes. This gives a direct measure of compression efficiency and forms the x-axis of the rate-distortion curve;
2. **Entropy Estimation and comparison:** The first-order entropy $H(X)$ will be estimated for each subband independently and compared against the achieved code length.
3. **Peak Signal-to-Noise Ratio:** Measures the pixel-level difference between the original and reconstructed image. A higher PSNR means less distortion. PSNR will be computed at each quantization level to track quality degrades as the bitrate decreases;

4. **Subjective quality assessment:** Reconstructed images will be visually inspected at low, medium and high bitrates. The goal is to identify perceptual artefacts typical of wavelet-based codes at low bitrates;
5. **Rate-Distortion performance:** PSNR will be plotted as a function of bitrate, producing rate-distortion curves for each test image. These curve summarise the trade-off between compression and quality across the full range of quantization levels and will be compared against the DCI operating point.

B Implementation Details

This appendix describes the key components of the MATLAB implementation of the JPEG 2000-inspired implementation code. Each section presents the the relevant code excerpt followed by a detailed explanation.

B.1 Forward and Inverse Discrete Wavelet Transform

The forward DWT decomposes the image into a multi-resolution pyramid of subbands. At each level, the current approximation is filtered separately along rows and columns using the biorthogonal 4.4 wavelet (`bior4.4`), which is the filter specified by the JPEG 2000 standard [3]. This produces four subbands per level: *LL* (approximation), *LH* (horizontal edges), *HL* (vertical edges), and *HH* (diagonal edges). Only the *LL* subband is further decomposed in subsequent levels.

```
1 function [subbands, bookkeeping] = dwt2d_forward(img, num_levels)
2     dwtmode('per');           %Periodic extension to avoid border artefacts
3     filter = 'bior4.4';      % Biorthogonal 4.4 wavelet (used in JPEG 2000)
4     [LO_D, HI_D] = wfilters(filter, 'd'); %Decomposition filters (lowpasss
5                                     & highpass)
6
7     bookkeeping.sizes = cell(num_levels,1); %Store sizes for correct inverse
8                                     transform
9     bookkeeping.num_levels = num_levels;
10    subbands.LH = cell(num_levels,1);
11    subbands.HL = cell(num_levels,1);
12    subbands.HH = cell(num_levels,1);
13    current = img;
14    for j = 1:num_levels
15        [rows, cols] = size(current);
16        bookkeeping.sizes{j} = [rows cols];
17
18        %Horizontal 1D DWT
19        L = zeros(rows, cols/2);
```

```

19     H_arr = zeros(rows, cols/2);
20     for r = 1:rows
21         [L(r,:), H_arr(r,:)] = dwt(current(r,:), LO_D, HI_D);
22     end
23
24     %vertical 1D DWT
25     LL= zeros(rows/2, cols/2);
26     LH= zeros(rows/2, cols/2);
27     HL_arr = zeros(rows/2, cols/2);
28     HH= zeros(rows/2, cols/2);
29     for c = 1:cols/2
30         [LL(:,c), LH(:,c)] = dwt(L(:,c), LO_D, HI_D);           %Low
31                                     columns
32         [HL_arr(:,c), HH(:,c)] = dwt(H_arr(:,c), LO_D, HI_D); %High
33                                     columns
34     end
35     %Store detail subbands
36     %pass LL down to the next level
37     subbands.LH{j} = LH;
38     subbands.HL{j} = HL_arr;
39     subbands.HH{j} = HH;
40     current = LL;
41 end

```

The transform is applied separably: first along every row to produce low-frequency (L) and high-frequency (H) column arrays, and then along every column of each array. Periodic boundary extension (`dwtmode('per')`) is used to avoid border artefacts and to ensure the subband sizes halve exactly at each level, which simplifies bookkeeping during the inverse transform.

The inverse DWT reconstructs the image level by level, from the coarsest resolution back to the original size, using the stored subband sizes to handle boundary conditions correctly:

```

1 function img_rec = dwt2d_inverse(subbands, bookkeeping, num_levels)
2     filter = 'bior4.4';
3     current = subbands.LL;
4     for j = num_levels:-1:1
5         sz = bookkeeping.sizes{j};
6         current = idwt2(current, subbands.LH{j}, subbands.HL{j}, subbands.HH
7             {j}, filter, sz);
8     end
9     img_rec = current;
end

```

The size vector `sz` is passed to `idwt2` to ensure the reconstructed subband at each level matches the dimensions of the original input to that level, avoiding off-by-one errors introduced by the periodic extension.

B.2 Quantization

Deadzone scalar quantization is applied independently to each subband. The quantizer maps a coefficient x to an integer index q as:

$$q = \text{sgn}(x) \left\lfloor \frac{|x|}{\Delta} \right\rfloor \quad (1)$$

where Δ is the step size. Coefficients in the interval $(-\Delta, \Delta)$ map to zero, forming the dead zone that efficiently suppresses small noisy coefficients. Pre-subband step sizes are applied during the quantization loop:

```
1 function q = deadzone_quantize(x, delta)
2     q = sign(x) .* floor(abs(x) ./ delta);
3 end
```

```
1 function x_hat = deadzone_dequantize(q, delta)
2     x_hat = zeros(size(q));
3     mask = (q ~= 0);
4     x_hat(mask) = sign(q(mask)) .* (abs(q(mask)) + 0.5) .* delta;
5 end
```

During dequantization, non-zero indices are reconstructed at the midpoint of their quantization interval, $(|q| + 0.5) \Delta$, which minimizes the expected reconstruction error under a uniform distribution of coefficients within each bin.

Step sizes are assigned per subband and per decomposition level to reflect the perceptual importance and energy distribution of each subband:

```
1 function [subbands_q, stepSizes] = quantize_subbands(subbands,
2     delta_base, num_levels)
3     stepSizes.LL = delta_base * 0.5; % Finer quantization for the
4         approximation subband
5     for j = 1:num_levels
6         scale = 2^(j-1); % Scale increases with
7             decomposition level
8         stepSizes.LH{j} = delta_base * scale;
9         stepSizes.HL{j} = delta_base * scale;
10        stepSizes.HH{j} = delta_base * scale * 1.5;
11    end
12    subbands_q.LL = deadzone_quantize(subbands.LL, stepSizes.LL);
13    for j = 1:num_levels
```

```

11         subbands_q.LH{j} = deadzone_quantize(subbands.LH{j}, stepSizes.
12             LH{j});
13         subbands_q.HL{j} = deadzone_quantize(subbands.HL{j}, stepSizes.
14             HL{j});
15         subbands_q.HH{j} = deadzone_quantize(subbands.HH{j}, stepSizes.
16             HH{j});
17     end
18 end

```

The *LL* subband receives a finer step size (0.5δ) because it carries the low-frequency approximation of the image and dominates perceptual quality. At higher decomposition levels the subbands represent increasingly coarse detail, so their step sizes are scaled by 2^{j-1} . The *HH* subbands (diagonal edges) are quantized with a factor of $1.5\times$ relative to *LH* and *HL*, since diagonal features are less visually salient.

B.3 Adaptive Arithmetic Coding

All quantized subband coefficients are concatenated into a single symbol sequence, re-indexed to start at 1, and terminated with a dedicated EOF symbol before encoding:

```

1  function bitstream = arithmetic_encode_subbands(subbands_q, num_levels)
2      seq = [subbands_q.LL(:)];
3      for j = 1:num_levels
4          seq = [seq;
5              subbands_q.LH{j}(:);
6              subbands_q.HL{j}(:);
7              subbands_q.HH{j}(:)];
8      end
9      seq = int32(seq);
10     minSym = min(seq);
11     seq = seq - minSym + 1; % Shift symbols to start at 1
12     EOF_symbol = max(seq) + 1; % Reserve a symbol to signal end-of-
13     stream
14     seq = [seq; EOF_symbol];
15     alphabet_size = double(max(seq));
16     bitstream = arithmetic_encoder(seq, alphabet_size);
17 end

```

The encoder implements 32-bit integer arithmetic coding with an adaptive frequency model. After each symbol is coded, its frequency count is incremented, so the probability model tracks the empirical distribution of the input. The core of the encoder narrows the coding interval [*low*, *high*) for each symbol and then rescales it using three conditions:

```

1  function bitstream = arithmetic_encoder(data, alphabet_size)
2      precision = 32;
3      MAX_RANGE = 2^precision - 1;

```

```

4      HALF      = 2^(precision-1);
5      QUARTER   = 2^(precision-2);
6      THREE_QTR = 3 * QUARTER;
7      low  = uint64(0);
8      high = uint64(MAX_RANGE);
9      pending_bits = 0; %counts bits deferred due to E3 scaling
10     freq = ones(1, alphabet_size); % Initial uniform frequency counts (
        Laplace smoothing)
11     bitstream = [];
12     for k = 1:length(data)
13         symbol = data(k);
14         cumfreq = [0 cumsum(freq)];
15         total = cumfreq(end);
16         %narrow the interval to the sub-interval of the current symbol
17         range = double(high - low + 1);
18         high = low + floor(range * cumfreq(symbol+1) / total) - 1;
19         low  = low + floor(range * cumfreq(symbol) / total);
20         %Output bits and rescale while the interval lies in one half or
21         %converges toward the midpoint (E1, E2 E3 conditions)
22         while true
23             if high < HALF
24                 %interval in lower half: output 0
25                 [bitstream, pending_bits] = output_bit(bitstream, 0,
                    pending_bits);
26             elseif low >= HALF
27                 %interval in upper half: output 1
28                 [bitstream, pending_bits] = output_bit(bitstream, 1,
                    pending_bits);
29                 low  = low - HALF;
30                 high = high - HALF;
31             elseif low >= QUARTER && high < THREE_QTR
32                 %E3: interval straddles mid-point
33                 pending_bits = pending_bits + 1;
34                 low  = low - QUARTER;
35                 high = high - QUARTER;
36             else
37                 break;
38             end
39             low  = low * 2;
40             high = high * 2 + 1;
41         end
42         %update frequency count for adaptive model
43         freq(symbol) = freq(symbol) + 1;
44         %Halve counts if total grows too large

```

```

45         if sum(freq) > 1e5
46             freq = ceil(freq / 2);
47         end
48     end
49     %flush remainin bits: output the final distinguishing bit
50     pending_bits = pending_bits + 1;
51     if low < QUARTER
52         [bitstream, pending_bits] = output_bit(bitstream, 0,
53             pending_bits);
54     else
55         [bitstream, pending_bits] = output_bit(bitstream, 1,
56             pending_bits);
57     end
58 end

```

The E1 and E2 conditions output a definite bit when the interval lies entirely in one half of $[0, 1)$. The E3 condition handles the case where the interval converges towards the midpoint without crossing into either half: the bit is deferred as a *pending bit* and later resolved with complementary bits via `output_bit`. This prevents precision underflow while maintaining correct decoding. Frequency counts are halved when their sum exceeds 10^5 to keep the model adaptive to local statistics without overflow.

B.4 Entropy estimation

To obtain a theoretical lower bound on the achievable bit rate, a first-order adaptive entropy estimate H_1 is computed for each subband. The context for each coefficient is a binary variable indicating whether the previous coefficient was non-negative.

Within each context $c \in \{0, 1\}$, the smoothed probability of each symbol s is estimated using Laplace (add-one) smoothing to avoid zero-probability issues:

$$\hat{p}(s | c) = \frac{n_{s,c} + \alpha}{N_c + \alpha A_c} \quad (2)$$

where $n_{s,c}$ is the count of symbol s in context c , N_c is the total number of symbols in context c , A_c is the alphabet size in that context, and $\alpha = 1$.

```

1     function bits_h1 = subband_h1(sb)
2         vals = double(sb(:));
3         alpha = 1;
4         %binary context: 1 if previous value >=0 else 0
5         ctx_bin = double([0; vals(1:end-1)] >= 0);
6         L_total = 0;
7         for c = 0:1
8             vals_c = vals(ctx_bin == c); %coefficients under context c
9             if isempty(vals_c)

```

```

10         continue;
11     end
12     symb = unique(vals_c);
13     N_c = numel(vals_c);
14     A = numel(symb);
15     for s = 1:A
16         n_ic = sum(vals_c == symb(s));
17         %smoothed probability estimate
18         p_hat = (n_ic + alpha) / (N_c + alpha * A);
19         L_total = L_total + n_ic * (-log2(p_hat)); %cross-entropy
20         %contribution
21     end
22     bits_h1 = L_total;
23 end

```

The total cross-entropy $L = -\sum_{s,c} n_{s,c} \log_2 \hat{p}(s|c)$ gives the estimated number of bits required by an ideal coder using this model. Comparing this value against the actual arithmetic coding output quantifies the redundancy of the arithmetic coder relative to the H_1 model.

B.5 Main processing loop

The main loop iterates over each quantization step size δ and executes the full codec pipeline: forward transform, quantization, entropy coding, dequantization, inverse transform, and quality evaluation. This structure allows a direct comparison of rate-distortion performance across all tested values of δ .

For each value of δ , the image is first decomposed into subbands via the forward DWT, then quantized and entropy-coded:

```

1  for k = 1:numel(delta_values)
2      delta = delta_values(k);
3      fprintf('delta = %d\n', delta);
4      % BLOCK T: WAVELET TRANSFORM
5      [subbands, bookkeeping] = dwt2d_forward(img, num_levels);
6      % BLOCK Q: QUANTIZATION
7      [subbands_q, stepSizes] = quantize_subbands(subbands, delta,
8          num_levels);
9      % ENTROPY CODING: ARITHMETIC CODING
10     bitstream = arithmetic_encode_subbands(subbands_q, num_levels);
11     bits_arith = length(bitstream);
12     bpp_arith_all(k) = bits_arith / (H * W);
13     cf_arith_all(k) = 8 / bpp_arith_all(k);

```

The bits per pixel (bpp) is computed by dividing the total bitstream length by the number of image pixels $H \times W$. The compression factor (CF) is defined as the ratio of the

uncompressed bit depth (8 bpp) to the compressed rate, i.e. $CF = 8 / \text{bpp}$.

In parallel with arithmetic coding, the H_1 cross-entropy estimate is computed to measure how close the coder operates to the theoretical limit of the first-order model:

```

1  % CROSS-ENTROPY ESTIMATION (ORDER-1 MODEL)
2  bits_h1 = estimate_entropy_adaptive(subbands_q, num_levels);
3  bpp_h1_all(k) = bits_h1 / (H * W);
4  cf_h1_all(k) = 8 / bpp_h1_all(k);
5  redundancy_bits = bits_arith - bits_h1;
6  redundancy_pct = 100 * redundancy_bits / bits_arith;

```

A positive redundancy indicates that the arithmetic coder produces more bits than the H_1 estimate predicts. This gap arises because the arithmetic coder uses a flat order-0 model (uniform initial frequencies), whereas the H_1 estimate exploits a binary context, giving it a tighter bound on the true entropy of the subband coefficients.

After entropy coding, the pipeline reconstructs the image through dequantization and the inverse DWT, and evaluates objective quality via MSE and PSNR:

```

1  % BLOCK Q^{-1}: DEQUANTIZATION
2  subbands_rec = dequantize_subbands(subbands_q, stepSizes, num_levels);
3  % BLOCK T^{-1}: INVERSE WAVELET TRANSFORM
4  img_rec = dwt2d_inverse(subbands_rec, bookkeeping, num_levels);
5  mse_all(k) = mean((img(:) - img_rec(:)).^2);
6  psnr_all(k) = compute_psnr(img, img_rec);

```

After all iterations, results are collected into a summary table and plotted as a rate-distortion curve, with both the arithmetic coding rate and the H_1 estimate displayed for comparison:

```

1  figure('Name','Rate-Distortion');
2  plot(bpp_arith_all, psnr_all, 'b-o', 'LineWidth',2, 'MarkerSize',8);
3  hold on;
4  plot(bpp_h1_all, psnr_all, 'r-s', 'LineWidth',2, 'MarkerSize',8);
5  %Annotate each point with its delta values
6  for k = 1:numel(delta_values)
7      text(bpp_arith_all(k), psnr_all(k)+0.7, sprintf('\delta=%d',
8          delta_values(k)), 'FontSize',7, 'HorizontalAlignment','center');
9      text(bpp_h1_all(k), psnr_all(k)-1.0, sprintf('\delta=%d',
10         delta_values(k)), 'FontSize',7, 'HorizontalAlignment','center', '
11         Color','r');
12 end
13 xline(1.5, '—k', 'DCI reference (1.5 bpp)');
14 yline(45, '—m', '45 dB');
15 xlabel('Rate (bits/pixel)');
16 ylabel('PSNR (dB)');
17 title('Rate-Distortion Performance');

```

The DCI reference line at 1.5 bpp corresponds to the maximum bit rate permitted for digital cinema distribution under the DCI specification [1]. The 45 dB horizontal line serves as a perceptual quality reference, above which compression artefacts are generally considered imperceptible.

C User Guide

C.1 Requirements

- MATLAB R2021b or later;
- Wavelet Toolbox (wfilters, dwt, idwt2, dwtmode);
- Image Processing Toolbox (imread, im2gray);
- Test Image: lenna.tif (512×512 , 8-bit grayscale).

C.2 Running the code

Place `final_code.m` and `lenna.tif` in the same directory. Run from the MATLAB Command Window:

```
1 >> run('final_code.m')
```

The script will:

1. Print a per $-\delta$ table with code length, entropy estimate, redundancy and PSNR;
2. Display reconstructed images for all values of δ (1,4,8,16,32,64,128);
3. Plot the rate-distortion curve;
4. Print the summary table.

C.3 Key Parameters

Parameter	Default	Description
<code>num_levels</code>	3	Number of DWT decomposition levels
<code>delta_values</code>	[4,8,16,32,64,128]	Quantization step sizes to evaluate
<code>filter</code>	'bior4.4'	CDF 9/7 wavelet filter
<code>alpha (H1)</code>	1	Laplace smoothing parameter for entropy estimation

Table 5: Configurable parameters

C.4 Output Interpretation

- $\text{bpp}(\text{arith})$: actual arithmetic code length in bits per pixel;
- $\text{bpp}(\text{H1})$: lower bound and adaptive first-order context model entropy estimate;
- Redundancy (%) : fraction of $\text{bpp}(\text{arith})$ that a better context model would recover;
- $\text{PSNR} > 45$ dB satisfies DCI quality requirements.

D Statement on use of AI Tools

The following AI tools were used in the execution of this project:

- GitHub Copilot: Used for MATLAB code during the implementation of some of the functions, specially on the adaptive entropy coding. All suggestions were reviewed, tested and modified as needed;
- Claude (Anthropic): Used for improving report structure, grammar and clarity of technical writing. No AI-generated content was used directly without review and verification against cited source material.

All MATLAB code was written, tested and validated. All numerical results in table 3 were produced by running the code on the actual test image. All references were independently verified.